

Outliers, Inconsistencies, and Data Quality

Finding the values that do not belong — and deciding what to do about them

Data Wrangling

Fall 2026

By the end of today you can:

1. **Define** an outlier and name its three types
2. **Detect** outliers three ways: z-score, IQR, modified z-score
3. **Explain** why the z-score fails on small samples
4. **Decide** whether to drop, cap, keep, or fix a value
5. **Repair** inconsistencies (units, duplicates, categories)
6. **Score** a dataset on six quality dimensions

Three stories we will solve:

1. **The Bison Half-Marathon**
A 47-minute half marathon
2. **The \$1 Apartment**
A typo that hides in plain sight
3. **The Ice Cream Truck**
An outlier no single column can see

Rule for today

Every flagged value gets *investigated*, not deleted.

What Is an Outlier?

One value can move everything

Fifteen half-marathon finishing times (minutes). Fourteen runners finished between 92 and 121. One finished in **47**.

	With 47	Without 47	Change
Mean	101.8	105.7	−3.9 min
Median	104.0	105.0	−1.0 min
Standard deviation	17.37	8.79	+98%

The damage is not where you think

The mean moved by 4 minutes. The *standard deviation nearly doubled*. Every confidence interval, every *t*-test, every “ ± 2 SD” rule built on this dataset is now wrong — and one single row did it.

Outlier

An **outlier** is an observation that lies at an abnormal distance from the other values in a dataset, given some definition of “distance” and “abnormal.”

Notice what this definition does *not* say:

- It does not say the value is **wrong**.
- It does not say the value should be **removed**.
- It does not give a single **threshold** — “abnormal” is a choice you make and must defend.

Two famous outliers

Kept: In 1985 the Antarctic ozone hole readings were so low that NASA’s software had been discarding them automatically as errors for years. They were real.

Removed: A hospital records a patient age of 999. That is a sentinel value for “unknown,” not a very old patient.

Three types of outliers

Point (global)

One value is extreme compared to the whole dataset.

A \$45,000 monthly rent in a list of student apartments.

Detected by: z-score, IQR, MAD

Contextual

The value is normal overall, but abnormal *in its context*.

28° C is fine in July. In January in DC it is an outlier.

Detected by: group-wise rules, residuals from a model

Collective

No single value is odd, but a *group* of them together is.

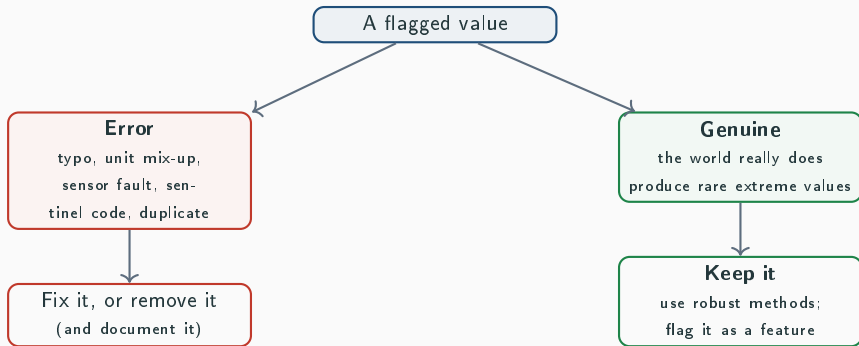
A heart rate of 65 is fine. Being flat at exactly 65 for 3 hours is not.

Detected by: sequence / pattern methods

Why this matters today

Our Example 3 is a **contextual** outlier. Every univariate method you are about to learn will look straight at it and report “nothing unusual.”

The fork in the road: where did it come from?



The question statistics cannot answer for you

"Is this value *possible*?" is a domain question. A 47-minute half marathon is physically impossible for an amateur. A \$45,000 rent is possible — in the wrong dataset.

Three Ways to Detect Them

$$z_i = \frac{x_i - \bar{x}}{s}$$

Rule of thumb: flag $|z_i| > 3$.

Reading it: z_i is how many standard deviations x_i sits from the mean. Under a normal distribution, only about 0.3% of values lie beyond ± 3 .

Assumes: roughly normal data, and a large enough sample.

```
z = (x - x.mean()) / x.std(ddof=1)
flagged = x[abs(z) > 3]
```

The hidden circularity

$\bar{x}$ and s are computed *from the data that contains the outlier*.

The outlier drags $\bar{x}$ toward itself and inflates s — and then we divide by that inflated s . The outlier helps to hide itself.

This is called **masking**, and Example 2 will show it doing real damage.

Method 1 has a hard ceiling

A fact most people never learn

For a sample of size n , the largest possible z-score is

$$|z|_{\max} = \frac{n-1}{\sqrt{n}}$$

No matter how extreme the value is. Not approximately — *exactly*.

n	$ z _{\max}$	Can the “ $ z > 3$ ” rule ever fire?
10	2.85	Never
12	3.18	Barely — only near the theoretical limit
20	4.25	Yes
50	6.93	Yes

Consequence

With $n \leq 10$, “no $|z| > 3$ ” tells you **nothing about your data**. It is a fact about arithmetic. Yet this is exactly the sample size of a typical class project.

Method 2 — The IQR rule (Tukey's fences)

Recipe

1. $Q_1 = 25\text{th percentile}$, $Q_3 = 75\text{th percentile}$
2. $\text{IQR} = Q_3 - Q_1$
3. Fences:
$$\text{lower} = Q_1 - 1.5 \times \text{IQR}$$
$$\text{upper} = Q_3 + 1.5 \times \text{IQR}$$
4. Flag anything outside the fences

Use $3.0 \times \text{IQR}$ for “far out” / extreme outliers.

```
q1, q3 = x.quantile(0.25), x.quantile(0.75)
iqr = q3 - q1
flagged = x[(x < q1 - 1.5*iqr) | (x > q3 + 1.5*iqr)]
```

Why it is better

Quartiles do not care how far away the outlier is. Move the 47 to 4.7 or to -1000 — Q_1 , Q_3 , and the fences *do not change at all*.

This is what a boxplot draws

The box is Q_1 to Q_3 ; the whiskers stop at the fences; the dots beyond are the flagged points. You have been running this method by eye for years.

Method 3 — The modified z-score (MAD)

Replace the mean with the **median** and the SD with the **median absolute deviation**:

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|)$$

$$M_i = \frac{0.6745 (x_i - \tilde{x})}{\text{MAD}}$$

Rule of thumb: flag $|M_i| > 3.5$.

The constant 0.6745 rescales MAD so that M_i is comparable to a z-score for normal data.

```
med = x.median()
mad = (x - med).abs().median()
m_score = 0.6745 * (x - med) / mad
flagged = x[m_score.abs() > 3.5]
```

Breakdown point

The fraction of your data that can be garbage before the method itself breaks:

Mean / SD	0%
IQR fences	25%
Median / MAD	50%

One bad value is enough to break the mean. Half your dataset must be corrupt to break the median.

Choosing a method

	z-score	IQR fences	Modified z (MAD)
Built from	mean, SD	quartiles	median, MAD
Breakdown point	0%	25%	50%
Assumes normality	yes	no	loosely
Small n (< 20)	fails	fine	fine
Skewed data	poor	fair	good
Multiple outliers	masking	robust	robust
Interpretation	“SDs from mean”	“outside the box”	“robust SDs”

Practical default

Start with a **boxplot**. Then run **IQR** and **modified z-score** together. If they disagree, that disagreement is information — go look at the actual rows.

Example 1 — The Bison Half-Marathon

Example 1 — The data

Setup. Fifteen runners in one age group. Times in minutes. The race director asks you to check the results before they are posted.

92, 95, 97, 98, 101, 103, 104, 106, 108, 110, 112, 115, 118, 121, 47

Sorted ($n = 15$):

47	92	95	97	98	101	103	104	106	108	110	112	115	118	121
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Positions: Q_1 at $\frac{n+1}{4} = 4$, median at $\frac{n+1}{2} = 8$, Q_3 at $\frac{3(n+1)}{4} = 12$.

Example 1 — Working the IQR rule by hand

Step 1 — Quartiles

$$Q_1 = 97 \quad Q_3 = 112$$

Step 2 — IQR

$$\text{IQR} = 112 - 97 = 15$$

Step 3 — Fences

$$\text{lower} = 97 - 1.5(15) = 74.5$$

$$\text{upper} = 112 + 1.5(15) = 134.5$$

Step 4 — Compare

$$47 < 74.5 \Rightarrow \text{FLAGGED}$$

Check against Python

NumPy interpolates quartiles differently:

	By hand	NumPy
Q_1	97	97.5
Q_3	112	111.0
IQR	15	13.5
Lower	74.5	77.25

Different numbers, **same conclusion**. Both flag 47 and nothing else.

And the verdict?

The fastest half marathon ever run is about 57 minutes. A 47-minute finish is not rare — it is **impossible**. This is an **error**: left branch, investigate and fix.

Example 2 — The \$1 Apartment

Example 2 — The data

Setup. Twelve monthly rents (\$) scraped from a listings site for apartments near campus. You are computing “average rent near campus” for a student housing report.

1800, 1950, 2000, 2100, 2150, 2200, 2250, 2300, 2400, 2500, 1, 45000

Two suspicious values

- **\$1** — a placeholder a landlord typed to make the listing appear in “lowest price” sorts
- **\$45,000** — an annual rent entered in a monthly field

The naive answer

$$\bar{x} = \$5,554.25$$

Ten of the twelve apartments rent for \$2,500 or less. The “average” is more than double every one of them.

Question: will the $|z| > 3$ rule catch them?

Example 2 — The z-score fails

$$\bar{x} = 5554.25, \quad s = 12439.24$$

Rent	z
1800	−0.302
2000	−0.286
2200	−0.270
2500	−0.246
1	−0.446
45000	+3.171

Two failures at once

1. **The \$1 is invisible.** Its z-score is -0.446 — more ordinary-looking than most of the real rents. *Masking.*

2. **The \$45,000 barely squeaks through.** $z = 3.171$, and for $n = 12$ the *maximum possible* z-score is

$$\frac{n-1}{\sqrt{n}} = \frac{11}{\sqrt{12}} = 3.175$$

So every z-score the rule can ever flag lives in the window $(3.000, 3.175)$ — a target 0.175 wide. This error landed at 99.9% of the ceiling.

Read that again

A value **45,000 times** larger than another value in the same column was almost too small to flag.

Example 2 — The modified z-score succeeds

Robust ingredients

$$\tilde{x} = 2175 \quad \text{MAD} = 200$$

$$M_i = \frac{0.6745(x_i - 2175)}{200}$$

Rent	M	
1800	−1.26	
2000	−0.59	
2200	+0.08	
2500	+1.10	
1	−7.33	FLAG
45000	+144.43	FLAG

Both caught, decisively

Not marginally. The \$1 scores -7.33 against a cutoff of 3.5, and the \$45,000 scores 144.

The IQR rule also works

$$Q_1 = 1987.5, Q_3 = 2325, \text{IQR} = 337.5$$

Fences: (1481.25, 2831.25)

Flags \$1 and \$45,000, keeps all ten real rents.

Corrected mean of the ten real rents: \$2,165

Example 3 — The Ice Cream Truck

Example 3 — The data

Setup. An ice cream truck parks outside the library. For 20 days the owner logs the temperature and units sold. She wants a model: how much should she stock tomorrow?

Day	Temp (C)	Sales	Day	Temp (C)	Sales
D01	15.0	210	D11	25.0	399
D02	16.0	233	D12	26.0	414
D03	17.0	241	D13	27.0	428
D04	18.0	248	D14	82.4	427
D05	19.0	274	D15	29.0	461
D06	20.0	282	D16	30.0	493
D07	21.0	319	D17	31.0	474
D08	22.0	360	D18	32.0	508
D09	23.0	345	D19	20.0	450
D10	75.2	361	D20	24.0	349

Two different problems are hiding in this table. One is an inconsistency. One is a contextual outlier.

Example 3 — Problem 1: the inconsistency

A validity rule catches it instantly

Column	Plausible range
temp_c	−20 to 45 °C
sales	0 to 2000 units

Violations: **D10 (75.2)** and **D14 (82.4)**

The fix — convert, do not delete:

$$C = \frac{(F - 32) \times 5}{9}$$

$$75.2 \rightarrow 24.0 \quad 82.4 \rightarrow 28.0$$

Why this is not an “outlier” problem

Nothing about the ice cream business is unusual on those days. The *measurement process* was inconsistent. Two rows used a different unit. Deleting them would throw away two perfectly good observations.

\$327 million

The Mars Climate Orbiter was lost in 1999 because one team used pound-force seconds and another used newton-seconds. Unit inconsistency is not a small-data problem.

Example 3 — Problem 2: the invisible outlier

D19: 20°C, 450 units sold. Run every univariate method on the sales column:

Method	Result for D19	Flagged?
Range check	$0 \leq 450 \leq 2000$	No
z-score	$z = 0.93$ (cutoff 3)	No
IQR fences	fences = (49.8, 663.8); 450 is inside	No
Boxplot	well inside the whiskers	No
Max sales in the data	508 units (D18) — and $450 < 508$	

Every tool from this lecture says the value is fine

And every tool is **wrong**. 450 units is perfectly ordinary — for a 29°C day. On a 20°C day, the truck sells about 282.

The outlier only exists in relation to the temperature column.

Example 3 — Finding it with residuals

Step 1. Fix the units first (D10, D14).

Step 2. Fit sales against temperature:

$$\widehat{\text{sales}} = -60.5 + 17.8 \times \text{temp_c}$$

Step 3. Residual = actual — predicted.

Step 4. Standardize the residuals and flag $|r| > 3$.

Day	Pred.	Resid.	r
D06 (20°)	296.3	−14.3	−0.39
D16 (30°)	474.8	+18.2	+0.50
D19 (20°)	296.3	+153.7	+4.21

Now it is obvious

Every other day sits within half a standard deviation of the line. D19 sits **4.2** standard deviations above it — it sold 154 more units than any 20°C day should.

So what happened on D19?

The owner checks her calendar: **it was Homecoming**. Ten thousand extra people walked past the truck. The value is **real**. It is not an error.

Example 3 — The verdict

The wrong move

Delete D19. The model gets a nicer R^2 . It also now predicts 296 units for next Homecoming — and the truck runs out of ice cream by noon.

The right move

Keep it, and explain it. Add an `event_day` column. Now D19 is not an unexplained outlier — it is the single most valuable row in the dataset, because it is the only evidence you have about what events do to demand.

The general principle

An outlier you cannot explain is a data quality problem.

An outlier you *can* explain is a **missing variable**.

Ask “what is different about this row?” before you ask “how do I remove this row?”

Handling What You Find

Five things you can do (and when)

Action	Use when	Cost / risk
Fix	You can recover the true value (unit conversion, re-read the source)	Best option. Requires access to the source.
Remove	Confirmed error, unrecoverable, few rows	Shrinks n; biases results if the errors are not random
Cap (winsorize)	Genuine extremes you want to limit, not erase	Distorts the tail; never do it silently
Transform	Right-skewed positive data (income, counts)	log compresses extremes; changes interpretation of every coefficient
Keep + flag	Genuine and meaningful	Usually correct ; add an indicator column

Non-negotiable

Whatever you choose: **record how many rows were affected and why**, and report your headline result *with and without* the change.

Inconsistencies and Data Quality

Inconsistencies: same meaning, different bytes

Type	What it looks like	Fix
Units	75.2 and 24.0 in one temperature column	Range rule, then convert
Category variants	"NY", "N.Y.", "new york", "New York "	<code>.str.strip().str.lower()</code> , then a mapping dictionary
Date formats	03/04/2026 — March 4th or April 3rd?	Parse with an explicit format; store ISO YYYY-MM-DD
Duplicates	Same order ID twice; same person, two spellings	<code>.duplicated()</code> on a key; fuzzy match on names
Cross-field logic	<code>qty * price != total;</code> <code>ship_date < order_date</code>	Write the rule as an assertion and test every row
Sentinels	999, -1, "N/A", 1900-01-01	Convert to real missing values (NaN)

The pattern

Outliers hide in *numbers*. Inconsistencies hide in *encodings* — and a statistical detector will never find them.

Assessing data quality: six dimensions

1. **Completeness** — how much is missing?

```
df.isna().mean()
```

2. **Validity** — does it obey the rules?

```
age in [0, 120]; % in [0, 100]
```

3. **Consistency** — do fields agree?

```
qty*price == total
```

4. **Uniqueness** — any duplicates?

```
df.duplicated(subset='id')
```

5. **Accuracy** — does it match reality?

needs an external source

6. **Timeliness** — is it current enough?

```
how stale is the newest row?
```

Make it a habit

Write a `quality_report(df)` function once.
Run it on *every* dataset before you analyze anything.

```
def quality_report(df):  
    return pd.DataFrame({  
        "dtype":      df.dtypes ,  
        "missing_%": df.isna().mean()*100 ,  
        "unique":     df.nunique() ,  
        "min":        df.min(numeric_only=True) ,  
        "max":        df.max(numeric_only=True) ,  
    })
```

The full version is in the notebook.

Accuracy is the hard one: the other five you can check from inside the file. Accuracy requires the outside world.

Practice Problems

Practice Problem 1 — The impossible exam

Scenario. You are the TA for a course scored out of 100. Here are 14 scores from one section:

72, 68, 91, 85, 77, 80, 88, 74, 95, 83, 105, -5, 79, 86

1. Write a **validity rule** for this column. Which values violate it?
2. Compute Q_1 , Q_3 , IQR, and the Tukey fences **by hand**. Does the IQR rule flag the same values your validity rule did?
3. The 105 turns out to be a legitimate score of 95 plus 10 bonus points. The -5 was a placeholder for “did not sit the exam.” **Handle each one** — and say which of the five actions from the handling table you chose and why.
4. Recompute the class mean before and after. Write the one sentence you would put in your report.

Practice Problem 2 — The clinic intake sheet

Scenario. Ten rows from a clinic's patient table.

id	name	age	visit_date	bp_systolic
P01	Ana Reyes	34	2026-03-11	118
P02	Ben Osei	51	2026-03-11	132
P03	Cara Lin	999	2026-03-12	121
P04	Dan Meyer	45	03/12/2026	0
P05	Ana Reyes	34	2026-03-11	118
P06	Eve Duarte	29	2026-03-13	240
P07	Femi Ade	62	2026-13-14	128
P08	Gus Park	-3	2026-03-14	119
P09	Hana Cruz	47	2027-03-15	125
P10	Ivan Petrov	38	2026-03-15	134

Find every problem. There are at least **seven**. For each one: name the **quality dimension** it violates, and state your fix.

Hint: two of the problems are invisible if you only look one column at a time.

Practice Problem 3 — The online store

Scenario. An e-commerce orders table. Every row *looks* fine column by column.

order	city	qty	unit_price	total	ship_date
A100	Washington	2	19.99	39.98	2026-04-02
A101	washington	1	45.00	45.00	2026-04-02
A102	Silver Spring	3	12.50	37.50	2026-04-03
A103	WASHINGTON	4	19.99	79.96	2026-04-01
A104	Silver Spring	2	30.00	66.00	2026-04-04
A105	Washington DC	1	99.99	99.99	2026-04-05
A103	WASHINGTON	4	19.99	79.96	2026-04-01

Order date for every row is 2026-04-02.

1. Group by `city` and count orders. Why is the answer wrong? Write the cleaning code.
2. Write a **cross-field assertion** linking `qty`, `unit_price`, and `total`. Which row fails?
3. One row ships *before* it was ordered. Find it.
4. One order ID appears twice. Is it a duplicate row, or a real second order? **How would you decide?**

Wrap-Up

What to carry out of this room

1. **Investigate, do not delete.** A flagged value is a question, not a verdict.
2. **The z-score is the weakest tool you have.** Zero breakdown point, and for $n \leq 10$ it is mathematically incapable of flagging anything.
3. **Prefer robust methods:** IQR fences and the modified z-score.
4. **Fix inconsistencies before detecting outliers.** Units first, statistics second.
5. **Some outliers are invisible one column at a time.** If nothing looks wrong, compare columns.
6. **An outlier you can explain is a missing variable.**
7. **Document every change** and report results with and without it.

Coming up

Notebook: `outliers_data_wrangling.ipynb` — all three examples, plus the practice problems with worked solutions.

Questions?

Open the notebook and let's break some data.